

AI Purchasing Agent — submission note

Repository: <https://github.com/jadonsanchay/purchasing-agent> **Author:** Sanchay Jadon · jadonsanchay@gmail.com

What it is

A full-stack AI purchasing agent for a quick-commerce buyer. Given a purchasing situation it investigates the data through tools, decides (accept / modify / reject / investigate / escalate), executes against a mock ERP, and validates the outcome. When reality disagrees with the plan it recovers within a bounded number of turns, then escalates to the buyer.

Approach in one line

The LLM investigates and proposes. Deterministic code gates, executes and verifies. The model has no write tools: every ERP change goes propose → policy check → optional human approval → execute → validate.

Scope

- Scenario 1 (recommendation review): end to end, six fixtures (baseline, open PO already covers demand, budget cap, storage cap, forecast vs actuals disagree, budget consumed during execution).
- Scenario 2 (supplier cannot fulfil): end to end, three fixtures (alternate available, partial is enough, alternate fails on execution → escalate).

Feedback loop and validation

The mock ERP enforces its own rules and can be made to disagree with the agent (supplier rejects the PO, budget consumed concurrently, partial confirmation). The validator diffs the agent's stated expected outcome and the system's invariants against the real post-write state; mismatches go back to the agent as feedback; after MAX_RECOVERY_TURNS the run escalates with a summary. Every step is in the audit log and the UI trace.

Evaluation

backend/evals/cases.yaml scores each run on the six questions from the brief (decision, information gathered, constraints, action, validation, recovery). The rubric is itself tested: it must pass nine reference trajectories and fail a blind accept, an over-buy on top of an open PO, an unneeded alternate order, and a skipped recovery. Live evaluation on gpt-4.1: 26/27 runs passed with 3 repeats per case (backend/evals/report.md). The scripted harness self-check is 9/9.

Running it

```
cp .env.example backend/.env      # add OPENAI_API_KEY, or LLM_PROVIDER=scripted for a no-key demo
make setup && make dev              # API :8000, UI :5173
make test                          # 55 tests, no network
```

Stack: Python 3.10, FastAPI, SQLite, OpenAI Responses API (strict function tools), pytest; React + Vite + TypeScript.
Architecture diagram: docs/architecture.md. Design trade-offs: docs/decisions.md.